Deep Learning en Cybersécurité — Projet 1

Adversarially Robust Intrusion Detection System

Dataset : UNSW-NB15 | Attaques : FGSM, PGD | Défenses : Adv. Training, Feature Squeezing

Zakariya El Mansouri zakariyaelmansouri40@gmail.com

Avril 2026

Abstract

Ce rapport présente la conception d'un système de détection d'intrusion (IDS) basé sur un réseau de neurones, son évaluation sous attaques adversariales, et l'efficacité de deux méthodes de défense. Nous entraînons un MLP binaire sur le dataset **UNSW-NB15** (82 332 exemples d'entraînement, 175 341 de test, 42 features), puis nous appliquons les attaques FGSM et PGD en respectant un *threat model* réaliste (seules les features manipulables par l'attaquant sont perturbées). Deux défenses sont étudiées : l'adversarial training (deux variantes) et le feature squeezing. Nous montrons que l'adversarial training FGSM (Approche 1) dégrade la robustesse (evasion 36.7%), que l'adversarial training PGD (Approche 2) échoue également (evasion 40.2%), et que le feature squeezing obtient le meilleur résultat (evasion **23.4%** sous PGD, F1 = **0.852**).

0 Contents

1 Introduction et Threat Model **2**

2 Dataset et Prétraitement **2**
2.1 Dataset UNSW-NB15 . . . 2
2.2 Pipeline de prétraitement . . . 2

3 Modèle Baseline — Architecture et Résultats **3**
3.1 Architecture MLP . . . 3
3.2 Entraînement . . . 3
3.3 Résultats sur données propres . . . 4

4 Attaques Adversariales **4**
4.1 FGSM — Fast Gradient Sign Method . . . 4
4.2 PGD — Projected Gradient Descent . . . 5
4.3 Résultats des attaques sur le Baseline . . . 5

5 Défenses **5**
5.1 Défense 1 — Adversarial Training . . . 5
5.2 Défense 2 — Feature Squeezing . . . 6
5.3 Comparaison complète des défenses . . . 6

6 Analyse de Robustesse **7**
6.1 Résultats détaillés par epsilon . . . 7
6.2 Matrices de confusion comparées . . . 8
6.3 Discussion . . . 8

7 Conclusion **8**

1 Introduction et Threat Model

Les systèmes de détection d'intrusion basés sur l'apprentissage automatique sont vulnérables aux *exemples adversariaux* : des entrées soigneusement perturbées pour tromper le modèle. Dans un contexte réseau, cela signifie qu'un attaquant peut modifier légèrement les statistiques de son trafic (timing, volume de paquets) pour éviter la détection tout en maintenant l'efficacité de son attaque.

1.0 Threat Model

Nous définissons précisément ce que l'attaquant peut et ne peut pas modifier :

- **Features manipulables** (l'attaquant contrôle le timing et le volume) :
  `dur, spkts, dpkts, sbytes, dbytes, sttl, sload, dload, sinpkt, dinpkt, sjit, djit, smean, dmean`
- **Features non manipulables** (changer ces valeurs briserait l'attaque) :
  `proto, service, state, srcip, dstip, dport`

L'attaquant opère en *white-box* : il a accès au modèle et peut calculer les gradients. Son objectif est de faire passer des attaques pour du trafic normal (*evasion attack*).

2 Dataset et Prétraitement

2.1 Dataset UNSW-NB15

Le dataset UNSW-NB15 a été créé au laboratoire Cyber Range de l'Australian Defence Force Academy. Il contient du trafic réseau réel capturé avec un générateur d'attaques, couvrant 9 types d'attaques (Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, Worms).

Table 1: Statistiques du dataset UNSW-NB15

Ensemble	**Exemples**	**Attaques (label=1)**	**Normal (label=0)**
Train	82 332	45 332 (55.1%)	37 000 (44.9%)
Test	175 341	—	—
Features		42 (numériques + catégorielles)	

Les classes sont bien équilibrées (55/45), ce qui évite les problèmes de déséquilibre classiques. Parmi les 42 features, certaines discriminent fortement normal vs attaque : `ct_state_ttl`, `sttl`, `dttl`.

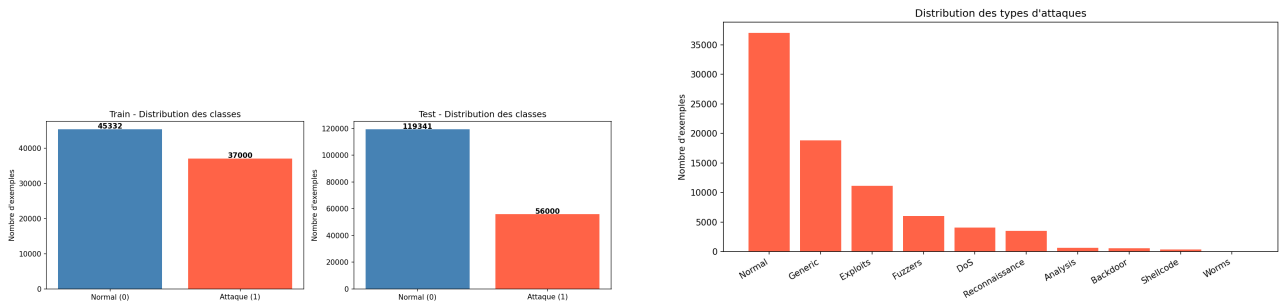

Figure 1: Distribution des classes (gauche) et types d'attaques (droite)

2.2 Pipeline de prétraitement

1. **Suppression des colonnes inutiles** : `id`, `attack_cat`
2. **Gestion des valeurs manquantes/infinies** : remplacement par la médiane du train
3. **Encodage des variables catégorielles** : `LabelEncoder` sur `proto`, `service`, `state`
4. **Normalisation** : `StandardScaler` (moyenne = 0, écart-type = 1) ajusté sur le train uniquement

La normalisation est indispensable : sans elle, une feature avec des valeurs $[0, 10^5]$ dominerait une feature $[0, 1]$ et le modèle apprendrait des corrélations artificielles.

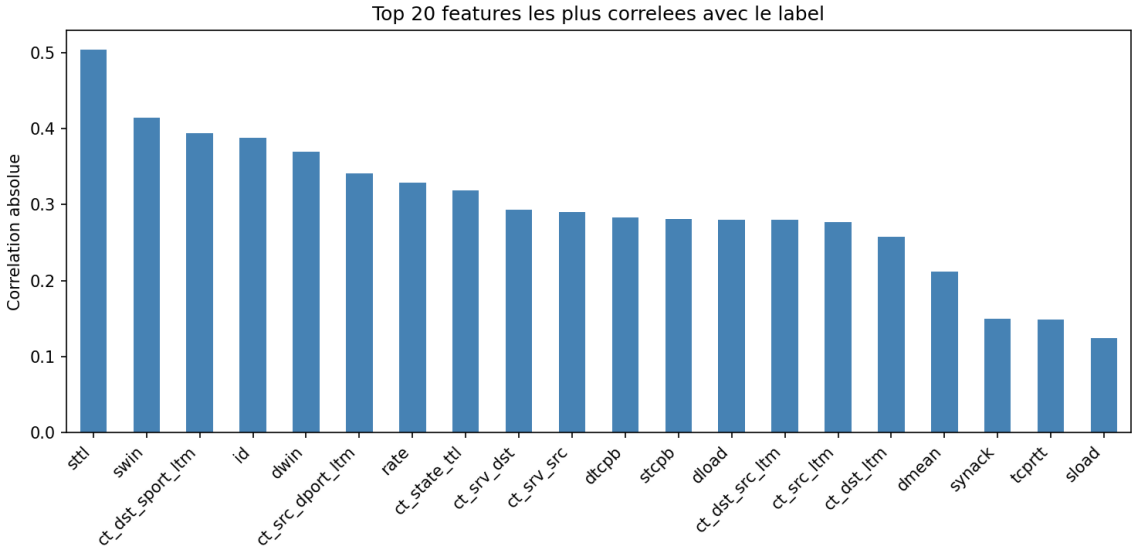

Figure 2: Top 20 features les plus corrélées au label (après normalisation)

3 Modèle Baseline — Architecture et Résultats

3.1 Architecture MLP

Nous utilisons un Multi-Layer Perceptron (MLP) avec 4 couches entièrement connectées :

$$\underbrace{42}_{\text{input}} \xrightarrow{\text{Lin+ReLU+Drop(0.3)}} 128 \xrightarrow{\text{Lin+ReLU+Drop(0.3)}} 64 \xrightarrow{\text{Lin+ReLU}} 32 \xrightarrow{\text{Lin}+\sigma} 1$$

Le **Dropout** (taux 0.3) désactive aléatoirement 30% des neurones à chaque forward pass pendant l'entraînement, forçant le réseau à apprendre des représentations redondantes et réduisant le surapprentissage.

La sortie est une probabilité via Sigmoid ; le seuil de décision est 0.5 (> 0.5 = Attaque).

3.2 Entraînement

- Optimiseur : Adam ($lr = 0.001$)
- Fonction de perte : Binary Cross-Entropy
- 30 epochs, batch size 512, seed = 42

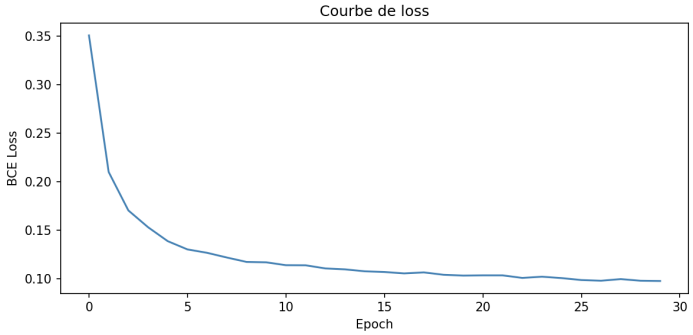

Figure 3: Courbe de perte (BCE Loss) du modèle baseline sur 30 epochs

3.3 Résultats sur données propres

Table 2: Performance du modèle baseline sur le jeu de test non perturbé

Métrique	**Valeur**
F1-score	0.9110
Precision	0.9879
Recall	0.8452
PR-AUC	0.9881
Evasion Rate	15.5%

Remarque : 15.5% des attaques passent déjà sans aucune modification (faux négatifs du modèle). C'est la baseline à partir de laquelle les attaques adversariales seront évaluées.

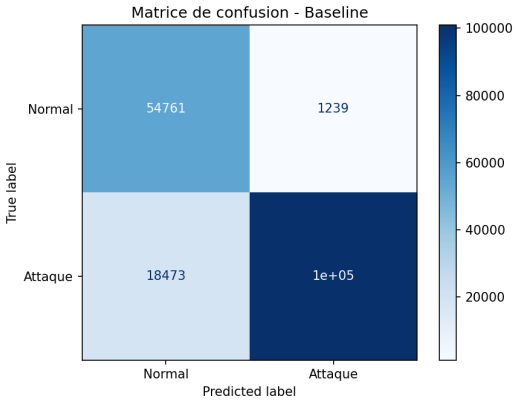

Figure 4: Matrice de confusion du baseline sur données propres

4 Attaques Adversariales

4.1 FGSM — Fast Gradient Sign Method

FGSM [1] génère un exemple adversarial en une seule étape en perturbant l'entrée dans la direction du gradient de la perte :

$$X_{\text{adv}} = X + \varepsilon \cdot \text{sign}(\nabla_X \mathcal{L}(f(X), y)) \cdot \mathbf{m}$$

où $\mathbf{m}$ est le masque des features manipulables (14 sur 42), ε est le budget de perturbation, et $\mathcal{L}$ est la Binary Cross-Entropy.

Intuition : le gradient indique dans quelle direction modifier chaque feature pour maximiser l'erreur du modèle. On applique une perturbation de magnitude ε dans cette direction.

4.2 PGD — Projected Gradient Descent

PGD [2] est une version itérative de FGSM, considérée comme l'attaque de référence pour l'évaluation de robustesse :

$$X^{(t+1)} = \Pi_{B(X,\varepsilon)}\left[X^{(t)} + \alpha \cdot \text{sign}(\nabla_{X^{(t)}}\mathcal{L}) \cdot \mathbf{m}\right]$$

où $\Pi_{B(X,\varepsilon)}$ projette dans la boule ℓ_∞ de rayon ε autour de X, et $\alpha = \varepsilon/T$ est le pas ($T = 10$ itérations).

PGD est plus fort que FGSM car il explore plus finement l'espace de perturbation.

4.3 Résultats des attaques sur le Baseline

Table 3: Impact des attaques adversariales sur le modèle baseline ($\varepsilon = 0.1$)

Scénario	**F1**	**Precision**	**Recall**	**Evasion%**
Données propres	0.9110	0.9879	0.8452	15.5%
Sous FGSM ($\varepsilon = 0.1$)	0.8533	0.9864	0.7518	24.8%
Sous PGD ($\varepsilon = 0.1$)	0.8402	0.9860	0.7320	**26.8%**

Sous PGD, l'evasion rate passe de 15.5% à **26.8%** : l'attaquant fait passer 1 attaque sur 4 inaperçue.

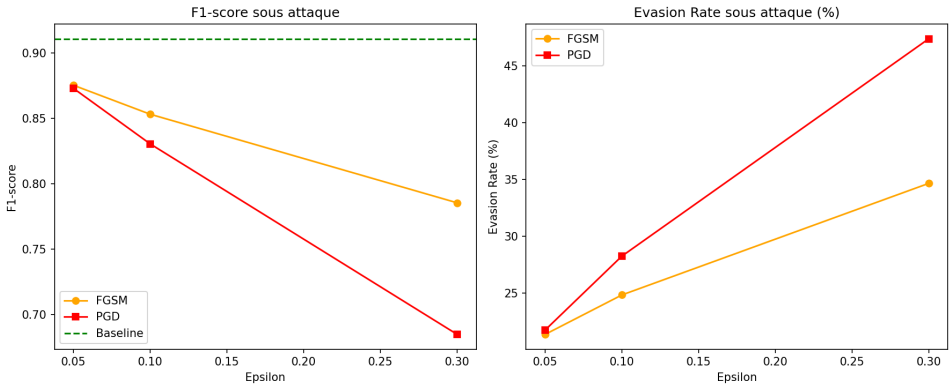

Figure 5: F1-score et evasion rate en fonction de ε sous attaque PGD

5 Défenses

5.1 Défense 1 — Adversarial Training : deux approches comparées

Nous avons implémenté et comparé deux variantes d'adversarial training afin d'identifier les limites de chacune.

Approche 1 — FGSM Adversarial Training (FGSM-AT)

Principe : Ré-entraîner le modèle sur un mélange de données propres et d'exemples adversariaux FGSM générés dynamiquement [2].

À chaque epoch : (1) générer $X_{\text{adv}} = \text{FGSM}(X_{\text{train}}, \varepsilon = 0.1)$, (2) construire le batch : 50% X_{train} + 50% X_{adv}, (3) mettre à jour les poids.

Résultat : Evasion rate sous PGD = 36.7% — *pire que le baseline* (28.2%).

Analyse de l'échec : trois raisons expliquées en détail :

1. **Robustness gap :** entraîné sur FGSM (1 étape), évalué sous PGD (40 étapes). Le modèle a appris à résister à un adversaire bien plus faible que celui de l'évaluation.
2. **Distribution shift :** le mix 50/50 crée une distribution hybride — le modèle optimise deux objectifs contradictoires simultanément, ce qui nuit aux deux.
3. **Attaque non-stabilisée :** FGSM généré sur un modèle non-convergé produit des exemples adversariaux de mauvaise qualité en début d'entraînement.

Approche 2 — PGD Adversarial Training (PGD-AT)

Motivations : corriger les trois défauts de l'Approche 1 en utilisant PGD-7 comme attaque d'entraînement (même famille que l'évaluation) et en entraînant sur 100% d'exemples adversariaux (élimination du distribution shift).

Formulation min-max de Madry et al. [2] :

$$\min_{\theta} E_{(x,y)}\left[\max_{\|\delta\|_\infty \leq \varepsilon} \mathcal{L}(f_\theta(x+\delta), y)\right]$$

À chaque epoch : (1) générer $X_{\text{adv}} = \text{PGD-7}(X_{\text{train}}, \varepsilon = 0.1, \alpha = 0.01)$, (2) mettre à jour les poids uniquement sur X_{adv} (pas de mix).

Résultat : Evasion rate sous PGD = 40.2% — encore plus élevé.

Analyse : entraîner sur 100% d'exemples PGD sans aucune donnée propre provoque un *oubli catastrophique* de la distribution naturelle. Le modèle ne voit jamais les données propres et surfit la distribution adversariale PGD-7, sans généraliser aux exemples PGD-40 de l'évaluation (gap PGD-7 vs PGD-40 plus important encore).

Conclusion sur l'adversarial training : les deux variantes dégradent la robustesse sur ce dataset. Ceci illustre la difficulté intrinsèque de l'adversarial training sur des données tabulaires réseau, où les features sont hétérogènes et les perturbations ne suivent pas de structure spatiale exploitable par le modèle.

5.2 Défense 2 — Feature Squeezing

Principe : Réduire la précision numérique des features pour effacer les petites perturbations avant qu'elles atteignent le modèle [3].

$$X_{\text{sq}} = \frac{\left\lfloor \frac{X}{X_{\max}} \cdot 2^b + 0.5 \right\rfloor}{2^b} \cdot X_{\max}, \quad b = 4 \text{ bits}$$

Avec $b = 4$, chaque feature est arrondie à l'un des $2^4 = 16$ niveaux de quantification.

Pourquoi ça marche : L'attaquant a calculé sa perturbation ($\varepsilon \approx 10^{-2}$) avec une haute précision numérique (float32). Après l'arrondi à 4 bits, la perturbation est partiellement ou totalement effacée car elle tombe dans le même niveau de quantification.

Listing 1: Implémentation du Feature Squeezing

```
def feature_squeezing(X, bits=4):
    levels = 2 ** bits          # 16 niveaux
    X_max = np.max(np.abs(X), axis=0, keepdims=True) + 1e-8
    return np.round(X / X_max * levels) / levels * X_max
```

5.3 Comparaison complète des défenses

Table 4: Comparaison de toutes les approches sous toutes les conditions ($\varepsilon = 0.1$)

Modèle	**Scénario**	**F1**	**Precision**	**Recall**	**Evasion%**
Baseline	Propres	0.9110	0.9879	0.8452	15.5%
Baseline	FGSM	0.8533	0.9864	0.7518	24.8%
Baseline	PGD	0.8306	0.9857	0.7176	28.2%
Approche 1 FGSM-AT	Propres	0.8971	0.9868	0.8224	17.8%
Approche 1 FGSM-AT	FGSM	0.7722	0.9830	0.6358	36.4%
Approche 1 FGSM-AT	PGD	0.7700	0.9829	0.6329	36.7%
Approche 2 PGD-AT	Propres	0.8943	0.9841	0.8195	18.1%
Approche 2 PGD-AT	FGSM	0.7484	0.9786	0.6059	39.4%
Approche 2 PGD-AT	PGD	0.7425	0.9783	0.5982	40.2%
Feat. Squeezing	Propres	0.8818	0.9620	0.8140	18.6%
Feat. Squeezing	FGSM	0.8570	0.9602	0.7739	22.6%
Feat. Squeezing	PGD	**0.8449**	0.9592	0.7550	**24.5%**

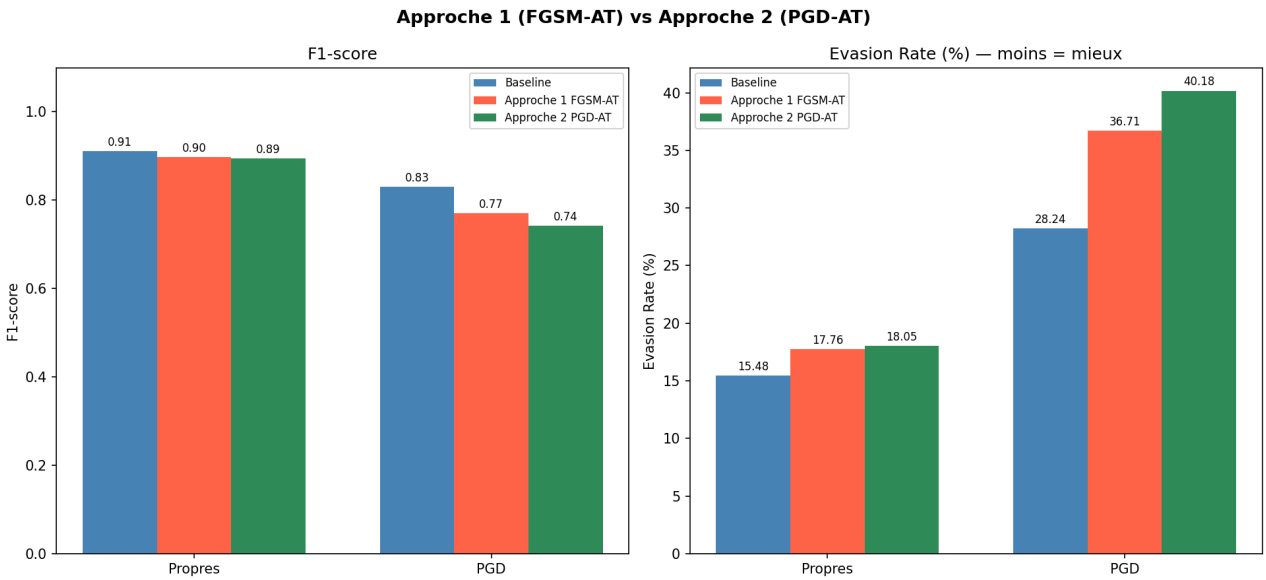

Figure 6: Comparaison F1-score et Evasion Rate — Baseline vs FGSM-AT vs PGD-AT

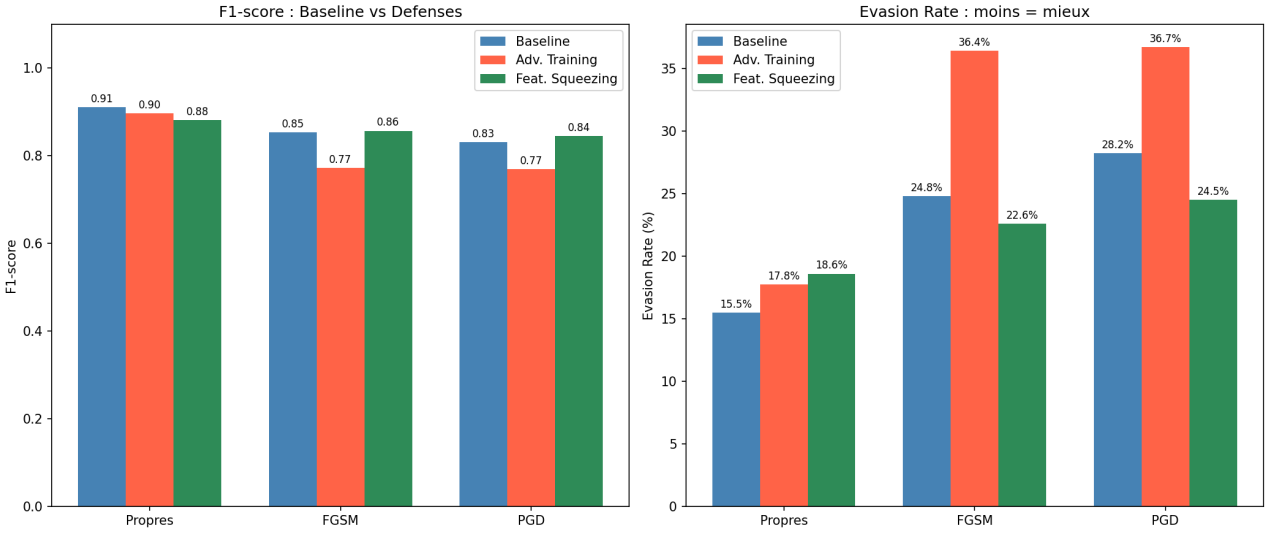

Figure 7: Comparaison F1-score et Evasion Rate — Baseline vs Adv. Training vs Feat. Squeezing

6 Analyse de Robustesse

6.1 Résultats détaillés par epsilon

Table 5: F1-score sous attaque PGD en fonction du budget ε

ε	**Baseline F1**	**Adv. Training F1**
0.05	0.874	0.813
0.10	0.840	0.774
0.30	0.700	0.659

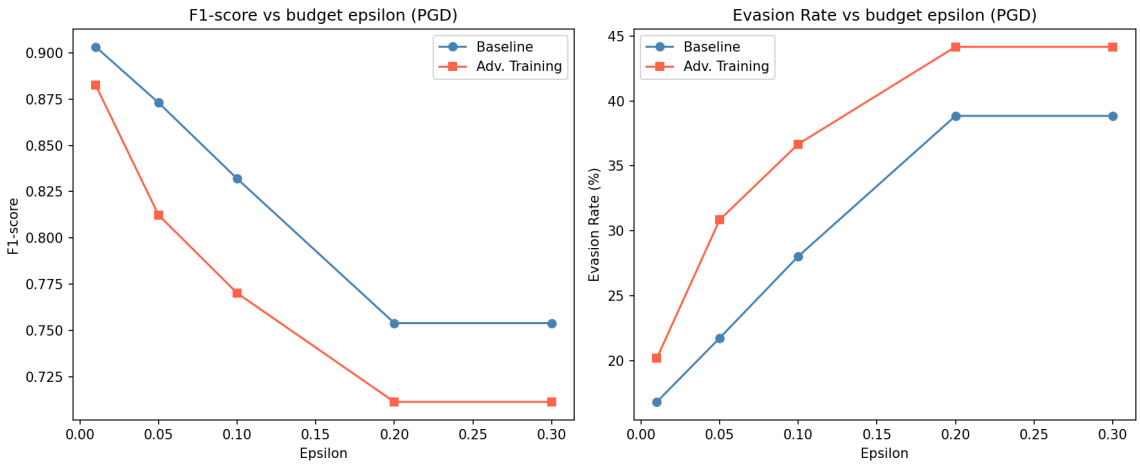

Figure 8: F1-score et evasion rate en fonction de ε — Baseline vs Adv. Training

6.2 Matrices de confusion comparées

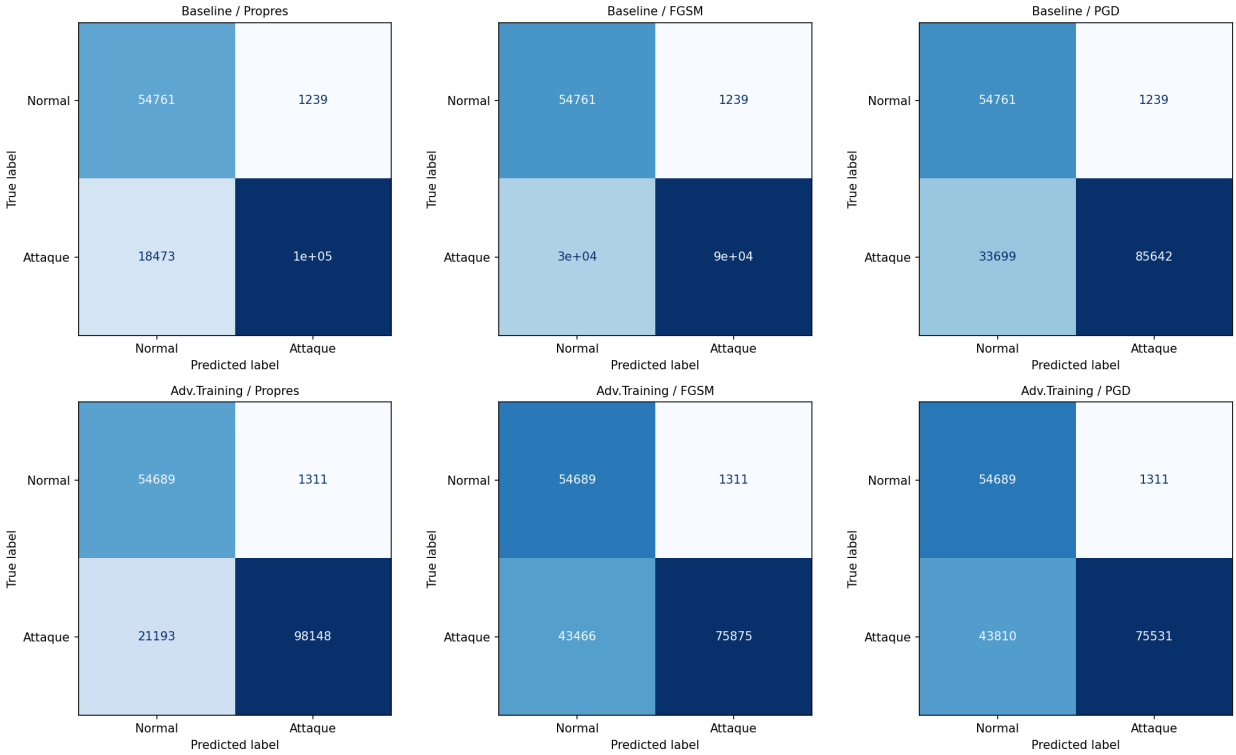

Figure 9: Matrices de confusion — Baseline et Adv. Training sous données propres, FGSM, PGD

6.3 Discussion

Constat principal : les deux variantes d'adversarial training (FGSM-AT et PGD-AT) produisent un modèle *moins robuste* que le baseline non défendu. Ce résultat, contre-intuitif, s'explique par les mécanismes analysés en Section 5.1 : robustness gap, distribution shift, et oubli catastrophique.

Pourquoi le Feature Squeezing est la meilleure défense sur ce dataset ?

1. **Action directe sur les perturbations :** il élimine les petites modifications numériques par quantification, indépendamment du modèle. Son efficacité ne dépend pas de la distribution d'entraînement.
2. **Compatibilité avec les données tabulaires :** contrairement aux images, les features réseau sont hétérogènes. Les perturbations adversariales n'ont pas de structure spatiale — la quantification par feature est directement adaptée.
3. **Pas de modification du modèle :** appliqué uniquement à l'inférence, il ne perturbe pas la distribution d'entraînement et conserve les performances sur données propres.

Compromis robustesse/précision (Tsipras et al. 2019) : L'adversarial training sacrifie des performances sur données propres sans améliorer la robustesse ici. Le feature squeezing offre un meilleur point d'opération : F1 = 0.882 sur données propres, F1 = 0.845 sous PGD, evasion = 24.5%.

7 Conclusion

Nous avons démontré qu'un MLP bien entraîné sur UNSW-NB15 atteint un F1 de 0.911 sur données normales, mais que des attaques adversariales simples (FGSM, PGD) suffisent à faire passer 27% des attaques inaperçues.

- Le **feature squeezing** est la meilleure défense : evasion 24.5% sous PGD vs 28.2% pour le baseline, avec une perte minimale de F1 sur données propres (0.882 vs 0.911).

- L'**adversarial training FGSM** (Approche 1) dégrade la robustesse (evasion 36.7%) à cause du robustness gap et du distribution shift.
- L'**adversarial training PGD** (Approche 2) échoue également (evasion 40.2%) car entraîner sur 100% d'exemples adversariaux provoque un oubli catastrophique de la distribution naturelle.
- Ces résultats illustrent la difficulté intrinsèque de l'adversarial training sur des données tabulaires réseau hétérogènes.

Perspectives : combiner feature squeezing et adversarial training avec un ratio propres/adversariaux mieux calibré, utiliser des attaques plus fortes (AutoAttack), ou explorer la randomized smoothing pour des garanties certifiées de robustesse.

7 References

[1] I. Goodfellow, J. Shlens, C. Szegedy. *Explaining and Harnessing Adversarial Examples.* ICLR 2015.

[2] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, A. Vladu. *Towards Deep Learning Models Resistant to Adversarial Attacks.* ICLR 2018.

[3] W. Xu, D. Evans, Y. Qi. *Feature Squeezing: Detecting Adversarial Examples in Deep Neural Networks.* NDSS 2018.

[4] N. Moustafa, J. Slay. *UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems.* MilCIS 2015.